

ENGINEERING REBOOT 2026

Week 08 - Day 37 Daily Field Manual

Method Overriding & `super()`

Phase II - Modern Software Engineering Foundations

Mission Brief

Day 36 established the fundamental inheritance relationship:

```
Employee
|-- SalariedEmployee
`-- HourlyEmployee
```

The subclasses inherited common state and behavior from `Employee` while introducing small subclass-specific behaviors.

Today we encounter the next engineering problem:

What happens when a subclass needs additional state or needs to provide a specialized version of behavior inherited from its parent?

Day 37 introduces two mechanisms for solving that problem:

- **method overriding**, which allows a subclass to provide specialized behavior; and
- `super()`, which allows subclass implementation to reuse or extend behavior defined by its parent.

The objective is not to make the hierarchy larger.

The objective is to understand how subclasses **specialize** an existing abstraction without unnecessarily duplicating its implementation.

1. Learning Objectives

By the end of Day 37, you should be able to:

- explain method overriding;
 - override an inherited method in a subclass;
 - recognize which implementation Python invokes for a particular object;
 - override a subclass constructor when additional state is required;
 - use `super()` to invoke parent initialization;
 - distinguish between **replacing** inherited behavior and **extending** inherited behavior;
 - avoid duplicating initialization already owned by the base class;
 - determine whether state or behavior belongs in the parent or subclass.
-

2. Starting Point

Recall the simplified Day 36 relationship:

```
class Employee:
    def __init__(self, employee_id, name):
        self.employee_id = employee_id
        self.name = name

    def display_info(self):
        print(self.employee_id, self.name)

class SalariedEmployee(Employee):
```

pass

SalariedEmployee automatically receives the constructor and `display_info()` behavior defined by Employee.

That works while every employee requires exactly the same state.

But suppose a salaried employee must also contain:

`annual_salary`

while an hourly employee requires:

`hourly_rate`
`hours_worked`

The inherited `Employee.__init__()` alone is no longer sufficient.

This creates today's first engineering problem.

3. Guided Exercise 1 - Extending Initialization with `super()`

Suggested filename

`employee_initialization.py`

Start with the existing Employee base class.

Create a SalariedEmployee subclass requiring:

- `employee ID`;
- `name`;
- `annual salary`.

A naive implementation might duplicate the parent initialization:

```
class SalariedEmployee(Employee):
    def __init__(self, employee_id, name, annual_salary):
        self.employee_id = employee_id
        self.name = name
        self.annual_salary = annual_salary
```

This works.

But now both Employee and SalariedEmployee contain responsibility for initializing:

`employee_id`
`name`

That duplication is unnecessary.

Instead:

```
class SalariedEmployee(Employee):
    def __init__(self, employee_id, name, annual_salary):
        super().__init__(employee_id, name)
        self.annual_salary = annual_salary
```

Observe

`super().__init__(employee_id, name)` delegates initialization of the common employee state to Employee.

The subclass then initializes only the state that belongs specifically to SalariedEmployee.

Exercise

Implement:

```
Employee
|-- SalariedEmployee
`-- HourlyEmployee
```

Use:

```
Employee
    employee_id
    name

SalariedEmployee
    annual_salary

HourlyEmployee
    hourly_rate
    hours_worked
```

Both subclasses should use `super()` for common initialization.

Checkpoint

Be able to explain:

The base class remains responsible for common Employee state. Each subclass initializes only the additional state required by its specialization.

4. Guided Exercise 2 - Method Overriding

Suggested filename

`method_overriding.py`

Suppose `Employee` provides:

```
def display_info(self):
    print(f"Employee ID: {self.employee_id}")
    print(f"Name: {self.name}")
```

A salaried employee needs additional information displayed.

The subclass can override the method:

```
class SalariedEmployee(Employee):

    def display_info(self):
        super().display_info()
        print(f"Annual Salary: {self.annual_salary}")
```

The method name remains:

`display_info()`

but the subclass supplies a specialized implementation.

Exercise

Provide specialized `display_info()` implementations for:

- `SalariedEmployee`;
- `HourlyEmployee`.

Each implementation should:

- 1 reuse the common `Employee` display behavior;
- 2 add information specific to that employee type.

5. Observe Method Resolution

Create:

```
employee = Employee(...)
salaried_employee = SalariedEmployee(...)
hourly_employee = HourlyEmployee(...)
```

Call:

```
employee.display_info()
salaried_employee.display_info()
hourly_employee.display_info()
```

Observe which implementation executes.

Ask:

- Why does `employee.display_info()` execute the `Employee` implementation?
- Why does `salaried_employee.display_info()` execute the `SalariedEmployee` implementation?
- What role does `super().display_info()` play inside the subclass?
- What would happen if the subclass did not override `display_info()`?

The important observation is:

Python selects behavior based on the object's class and its inheritance hierarchy.

Tomorrow, Day 38 will build directly on this behavior when we introduce **polymorphism**.

6. Replace vs. Extend

Method overriding does not automatically mean the parent implementation must be called.

A subclass can completely replace inherited behavior:

```
def some_method(self):
    # entirely specialized implementation
```

or extend it:

```
def some_method(self):
    super().some_method()
    # additional subclass behavior
```

The engineering question is:

Does the parent's behavior remain part of the subclass's intended behavior?

If yes, extending it with `super()` may be appropriate.

If no, the subclass may provide a completely specialized implementation.

Do not call `super()` mechanically merely because inheritance exists.

7. Mini-Project - Specialized Employee Hierarchy

Suggested files

A straightforward implementation may use:

```
employee.py
main.py
```

Additional files are not required unless they genuinely improve clarity.

Objective

Extend the Day 36 Employee hierarchy so each employee type maintains common Employee state while also maintaining state and behavior appropriate to its specialization.

Functional Requirements

Employee

The base class must maintain common employee information such as:

- employee ID;

- name.

It must provide at least one common behavior that subclasses can inherit or extend.

SalariedEmployee

Must inherit from Employee.

It must:

- initialize common Employee state using `super()`;
- maintain an `annual_salary`;
- override at least one inherited method;
- demonstrate behavior specific to salaried employees.

HourlyEmployee

Must inherit from Employee.

It must:

- initialize common Employee state using `super()`;
- maintain an `hourly_rate`;
- maintain `hours_worked`;
- override at least one inherited method;
- demonstrate behavior specific to hourly employees.

8. Expected Behavior

The program should instantiate at least:

```
1 SalariedEmployee
1 HourlyEmployee
```

For each object, demonstrate:

- successful parent initialization;
- successful subclass initialization;
- inherited state;
- overridden behavior;
- subclass-specific state or behavior.

The resulting execution should make it obvious that common Employee responsibilities remain in Employee, while specialized responsibilities belong to the subclasses.

9. Scope Guardrails

Day 37 does **not** require:

- collections of mixed employee types;
- generic payroll processing;
- polymorphic processing loops;
- `isinstance()` demonstrations;
- abstract base classes;
- protocols;

- multiple inheritance;
- deep inheritance hierarchies;
- JSON or CSV persistence;
- menus;
- CRUD operations;
- databases;
- sophisticated validation;
- production payroll calculations.

Those features do not contribute directly to today's learning objective.

In particular:

Do not deliberately build the Day 38 polymorphism exercise today.

If polymorphic behavior becomes apparent naturally, note the observation and leave its deliberate exploration for Day 38.

10. Engineering Questions

While implementing the exercises and mini-project, consider:

- 1 Which attributes genuinely belong to every `Employee`?
- 2 Which attributes belong only to `SalariedEmployee`?
- 3 Which attributes belong only to `HourlyEmployee`?
- 4 Why should the subclasses call the parent constructor instead of duplicating its assignments?
- 5 When overriding a method, should the subclass replace or extend the parent's implementation?
- 6 Does using `super()` make responsibility ownership clearer?
- 7 What maintenance problem could occur if common initialization were duplicated across every subclass?

11. Notes.md Reflection

Capture the important technical observations from today's work.

Suggested topics:

- definition of method overriding;
- purpose of `super()`;
- constructor specialization;
- parent versus subclass responsibility;
- replacing versus extending inherited behavior;
- method resolution observations;
- problems caused by duplicated initialization.

Do not treat these prompts as questions that require lengthy essay responses.

Record the observations that were meaningful during implementation.

12. Journal.md Reflection

Consider the engineering perspective rather than merely the syntax.

Possible reflection:

How did today's implementation change your understanding of inheritance compared with Day 36?

Consider whether inheritance now appears less like simple code reuse and more like a way of distributing responsibilities across related object types.

13. Definition of Done

Day 37 is complete when:

- Guided Exercise 1 is completed and executed.
- Guided Exercise 2 is completed and executed.
- `SalariedEmployee` extends parent initialization using `super()`.
- `HourlyEmployee` extends parent initialization using `super()`.
- At least one inherited method is overridden appropriately.
- Parent behavior is reused where appropriate rather than unnecessarily duplicated.
- The Specialized Employee Hierarchy mini-project is completed and tested.
- Code and execution evidence are captured according to current Engineering Reboot standards.
- `Notes.md` is completed.
- `Journal.md` is completed.
- Required Git operations are performed and captured.
- Changes are committed and pushed using the established daily workflow.

Day 37 Exit Criteria

Before closing the session, you should be able to explain the following without referring to documentation:

Why override a method?

Because a subclass needs a specialized implementation of behavior defined by its parent.

Why use `super()`?

Because the subclass can delegate appropriate responsibility to its parent rather than duplicating the parent's implementation.

What belongs in the base class?

State and behavior genuinely common to the abstraction represented by the base class.

What belongs in the subclass?

State and behavior specific to that specialization.

DAY 37 ENGINEERING PRINCIPLE

Specialize what is different. Reuse what remains common.

Tomorrow - Day 38

Polymorphism

Day 38 will take the specialized employee objects created today and ask a different question:

Can consuming code operate on different employee types through common behavior without needing to know which concrete subclass it received?

That is where the Week 08 inheritance sequence begins paying off at the application-design level.